



Google Summer of Code



Google Summer of Code 2025 End Term Eval

Project Details

Project: Resonate

Idea: Resonate Version 3

Project Size: Large (350 Hours)

Personal Information

Contact Information:

Name: Madhav Gupta

Discord Username: M4dhav

GitHub Profile URL: <https://github.com/M4dhav>

Work Done during GSoC 2025

30+ PRs Merged across both Repositories

100+ Commits across both Repositories

50k+ Lines of Code Contributed

Improve UI and Stabilize Codebase

When I started working on my project, the main and dev branches were out of sync as some PRs were merged directly into dev and the histories also did not match. I rebased the histories, solved all merge conflicts, and ensured that the code was brought to a stable state from where we could move forward keeping a systematic flow in mind that would culminate in an initial deployment of Resonate to the Play Store. I also improved the UI of some screens, where the visual contrast made it difficult to see the content on the screen. Additionally, previously Resonate's repo included keys to a Firebase Project used internally by the Resonate team for FCM functionality, and this was a security risk. So I also removed this file from the tracking and the entire history of the repo. I also fixed issues in the pubspec file and upgraded AGP to support 8.x+ along with upgrading the Android Configuration from Groovy to Kotlin DSL. While iOS deployment was not a direct target, I also fixed issues with the app on the iOS side. In the backend repo as well, I made a lot of changes, as there were dependency version conflicts and issues causing builds to fail.

Internationalization Framework

Previously, Resonate was only available in English, and this was a major issue with App Accessibility, as we could not expect all of our users to speak English. To this end, I added an Internationalization framework to the app, such that any contributor could easily add support for his native/bilingual language. To do this I had to make an ARB file from all statically defined strings app wide, implement the localizations via flutter_localizations and then replace the strings in the entire app with the localized versions. I also added translations for the entire App into Hindi, and added documentation for future contributors to be able to easily contribute translations in their language.

Deployment to Google Play Store

Previously Resonate had never been publicly deployed on any App Store. I deployed Resonate on the Google Play Store via AOSSIE's developer account. For this I had to configure release signing for the app, write the store listings, fill out data safety and permission questionnaires, compile and upload App Bundles, and request Production Access. I also added Privacy Policy and CSAE Standards documents for the Play Store Listing as per requirements.

Complete Chat Functionality Implementation

Previously the Chat Functionality was only partly implemented, such that just the screens and UI were implemented and not even part of the User Flow. I added the respective collections to the backend, connected the app to these collections, wrote the business logic to read and send messages, wrote functions to send the notifications for these messages, wrote the code to handle these notifications on the frontend and added the buttons to enter these chats while joined in Rooms.

Revamp Pair Chat system

Previously the Pair Chat system only included an option for random matching with users, i.e: If you wanted to have a Pair Chat the only option available to you was to join a random matcher with no prediction of when you would get to pair up with someone and no control over who to pair up with. I revamped this entire system by adding an option for users to select and see the available users for a Pair Chat at any given point in time. I created the respective collections in the backend, I wrote the screens for this feature, and wrote the business logic to handle the selection and matching as well. I also included an option for users to go back to random matching after selecting the option to choose someone if they wanted to.

Add Rating System as Passive Moderation

Previously Resonate had no real Moderation system except the option to kick troublemakers from rooms and even that was not final or permanent, the user could just join back in. As an app's user age rating depends highly on moderation systems when interactions with strangers is involved, it was critical for Resonate to have a Rating System. To this end I implemented a rating system in the revamped Pair chat functionality where a User could rate his pair after their Pair chat, using the classic 5 Star Rating option. This rating would then be shown on the user's profile and also next to his details on the pair chat screen, whenever anyone tried to pair up with him. For this I added the necessary variables to the backend, and wrote code to display and modify this on the frontend.

Setup CI/CD Pipelines for automated builds

As Resonate was now deployed on the Play Store and at a stable, predictable state, it made sense to setup CI/CD pipelines for automated builds that could build the app on commits to the master branch and upload the builds to Internal Testing on Google Play so that this would take strain off maintainers to manually build the Apps. For this I had to write the workflow, then encode sensitive files like Firebase Config files into base64 which I then converted into the files at runtime in the workflow. The workflow included multiple steps, like setting up Java and Flutter on GitHub runners, then fetching dependencies, decompiling the necessary files from the environment variables, and then compiling builds and finally uploading them to the internal testing track on Google Play.

Implement Automated Software Tests

Resonate previously had no automated software tests. This had caused regressions in the past, where functionalities were broken by some changes that past maintainers missed. Automated Software Tests for Resonate would mean reducing risk of Human Error and lessening the burden on maintainers to test every PR manually. After discussion with my mentor, we decided that the best thing to do was add unit tests for key functionalities and controllers. However, this posed a unique challenge as many of the controllers in Resonate are currently deeply coupled with the UI, and as a result, are not easily testable. While the preferred solution would be to rewrite the controllers while decoupling them entirely, we agreed that

this was outside the scope of my GSoC Project, and added unit tests for the controllers and functions that could be tested effectively. I also wrote workflows to automatically run these tests on every PR and added the running of tests to the pre-existing automated build workflow.

Enhanced Creator Interactions

I added some ways users could better interact with creators on the Resonate App, namely being able to search for them via the search bar on the Explore Page, which previously only searched for Stories. Additionally I also implemented a feature allowing users to follow their Favorite Creators. To do this I implemented the respective collections in the backend, and to make the Follow feature more functional, I added a function on the backend to send notifications to all followers every time the creator uploaded a new Story. I also added a screen to view followers and a button to follow and unfollow the creator on the Profile Page. I also added the business logic for this feature.

Friends Functionality and VoIP Calling

I added a functionality for users to add each other as friends. This feature's main use case was to enable One-on-One calling, and it differed from Pair Chats in the sense that Pair Chats would only work if both users were online and searching for users to Pair with. But the direct calling functionality that implemented along with the VoIP Calling also included sending the user a call from a terminated state of the app, similar to how you receive WhatsApp Calls even when the app is exited. To do this I created the respective collections and Screens. For the room itself I used the existing structure of Pair Chats, as the call was essentially a Pair Chat Room with extra functionality, but I still had to write the functions to send notifications of calls through FCM and then code to handle these notifications encoded specifically to represent that it was an incoming call and show a persistent notification about the call to the user even in a terminated state. I also wrote a notification tap handler that defined how the user would enter the call if he accepted it from the notification. When we went to deploy this, I also had to fill out the permission form for getting the full screen notification intent permission.

Meilisearch Integration

Previously Resonate used Appwrite Queries for all search related functionalities. This was good enough for a foundational design, but at scale this would drastically impact performance of searches and also lacked advanced features for searches. I added the option to alternatively use Meilisearch, an Open Source AI Powered Search Engine with advanced features like Vector Search, for all these search related functionalities. To do this I self-hosted an instance of Meilisearch, changed the backend setup script to automate this self hosting for other contributors, wrote functions to automatically sync added documents to a Meilisearch Instance, wrote a function to sync all existing documents to a Meilisearch Instance (If there were documents before Meilisearch was integrated), and wrote code on the Client Side to use this new option for searches with advanced features like vector search. For ease of use, I implemented a flag through environment variables to set whether to use Meilisearch or the legacy Appwrite Queries. This feature is not currently available in the deployed version, as even though the code is present, AOSSIE as a Non-Profit OSS Organization does not have the revenue to host a Meilisearch Instance for use at the moment.

Add support for new Android Requirements and release new version

While not a part of my GSoC Project, I added support to Resonate for the new Android Play Store requirements of a 16KB Page Size. Without this Resonate would have been removed from the Play Store in November. To do this I had to edit Android specific configuration files. Alongside this, on discussion with my mentor, we agreed that it was time for another version of Resonate to be released, that had a lot of new features. To do this I merged the changes into main and the build workflow worked perfectly and uploaded a testing build to internal testing, which I then promoted and sent for Production Review. Soon, after approval, a new version of Resonate was released.

Improved Host Moderation for Rooms

Previously Resonate had no real control for moderators and Admins when it came to Rooms, and troublemakers could abuse the system to make the Audio Rooms unusable. While there was an option to Mute users, this wasn't implemented correctly and only blocked a user from using their Mic toggle, so if you muted a user who was already speaking, nothing would happen to him. And while there was a Kick option, it did not stop the user from joining back in. I fixed the Mute Option and added an option to Report a user, which would not only kick him from the room, but block him from joining it again. Also, this is an automated system, so if a user gets reported 5 times, his access to the app will be revoked and he will need to contact AOSSIE if he thinks that the ban was unjustified to get himself unblocked. This adds another layer of safety and trust for Users on the Resonate app. To do this, I added configuration on the backend to track reports against a

specific user, and also on the frontend to report a user, and to check during login if he is to be blocked from the app and then show him the respective screens.

Live Chapter Functionality

Previously creators in Resonate could only upload pre recorded chapters. As Resonate already had features like Audio Rooms, these features had the potential to be merged into a functionality for Live Chapters, wherein a creator could start live streaming a chapter, and a notification would be sent out to all his followers regarding this. They could then join the audio room, where the creator would be speaking while the audio was being recorded locally on his device. Then after the creator ends the room, he goes to another page where he fills in other information about the chapter like the picture and the captions, and this recording is uploaded as another chapter alongside all the rest of the chapters. To implement this functionality I implemented the backend collections, functions to create and join the rooms, UI Screens for the Rooms, and added a flow to access this feature in the existing Add Chapter flow. I also implemented local On Device Transcription for the recorded audio using OpenAI’s Whisper base running locally. This generates timestamped transcriptions which I then format into an LRC as the lyrics are expected to be. On the next page where the user reviews the information, he can view and edit these lyrics, but the transcriptions are reliable for the most part, and can be directly used as is.

Release New Version of Resonate with all changes from Project

My GSoC Project culminated with the Release of a new version of Resonate with all these changes from my Project. To do this I again only had to merge the changes into main and the workflow I had written handled the building and uploading of the App Bundle. I then promoted the build to Production and a new version was released

Acting as Repo Maintainer

Alongside all my tasks for GSoC, I also acted as the Repo maintainer for Resonate throughout my project’s period, and I was responsible for opening issue for new contributors, assigning issues, reviewing and merging PRs, and mentoring any contributors willing to contribute to Resonate during my period as Contributor. This is also something that I would actively be willing to contribute to in the foreseeable future.

PRs Merged before Mid Sem Eval (Frontend Repo)

Pull Request #	Pull Request Title	Pull Request Contributions
PR #481	Brings app to Stable State, Dependency and Config Updates, Various Bugfixes	App brought to a stable state All commits to dev and main over the past various months merged Android Configuration upgraded from Groovy to Kotlin DSL Release Builds signing added for Android Notification Configuration and Permission Updates for iOS Replaced audiotags with flutter_media_metadata because of an iOS Config Issue UI Improvements to some screens Changes to Firebase and FCM specific logic pertaining to Required FCM Configuration in Upcoming Rooms Time Checker Function causes execution to fail Resonate-Backend#88 Various bugfixesAppropriate Documentation updates
PR #483	Add App Wide Internationalization Framework	This PR adds a framework to add Translations to Resonate, by using flutter_localizations to display strings throughout the app. It also included translations for the strings into Hindi
PR #484	Add privacy policy for deployment on Google Play Store	This PR added a Privacy Policy which we linked to our Play Store deployment
PR #485	AGP Version changes and Pubspec fix	This PR makes changes to the AGP and Kotlin versions that Resonate uses to fix version mismatches and compatibility issues. Additionally, it fixes an issue introduced by #483 where a static path is used to reference the audio_metadata_reader dependency.
PR #487	docs: Add Translations.md for new contributors to add translations	This PR adds a TRANSLATIONS.md doc for new contributors

Pull Request #	Pull Request Title	Pull Request Contributions
PR #491	Upload CSAE Standards doc for Play Store Listing	This PR adds a CSAE Standards document to the Resonate Documentation as required by Play Store for publishing the application.
PR #493	Connect Chat Pages to backend and implement notification functionality	This PR connects the Chat Functionality to the backend and adds notification functionality integration. Fixes #492
PR #495	Implement Functionality to allow choice of Pair Chat and Rating System	This PR adds the functionality to allow users to choose who to Pair Chat with and implements a Rating System that allows users to rate who they have chatted with. Fixes #494
PR #505	Setup CI/CD to automatically compile and deploy builds to Internal Testing on Google Play	This PR sets up a CI/CD Workflow to automatically compile and push app builds to the Google Play Store into internal testing. Additionally, it updates the Build Number in <code>pubspec.yaml</code> and makes changes to fetch sensitive information from Env Variables in <code>constants.dart</code> . It also makes the necessary documentation updates. Fixes #504

PRs Merged before Mid Sem Eval (Backend Repo)

Pull Request #	Pull Request Title	Pull Request Contributions
PR #85	Bump Appwrite yml version to fix Authentication issue and fix spelling mistakes	Fixes #84 and #81
PR #87	Added necessary configuration to Match Maker function	Fixes #86
PR #89	Comment out FCM Specific Code to improve onboarding experience	Implements changes described in #88
PR #90	Add Config and Functions related to Chat Functionality Integration	Adds Backend changes related to AOSSIE-Org/Resonate#492 .Additionally updates dependencies, sdk, and cli versions
PR #91	Add Choose Users Functionality and Rating System Config	This PR Adds Backend Support for the issue AOSSIE-Org/Resonate#494

PRs Merged after Mid Sem Eval (Frontend Repo)

Pull Request #	Pull Request Title	Pull Request Contributions
PR #509	Partial Implementation for Automated Software Tests	This PR adds tests for ~50% of key app functionalities that can be efficiently tested using a combination of widget and unit tests. It also modifies controllers and adds constructors for them so that they can be mocked for said tests. Additionally, a workflow to automatically run said tests on all PRs is also added. Fixes #508
PR #512	Implement advanced Creator Interaction Functionality	This PR adds functionality like searching for Creators via the Explore page and Following/Unfollowing Creators as well as receiving updates for stories posted by Followed Creators. Fixes #511
PR #513	Add Configuration for OAuth to work with workflows	This PR fixes the OAuth issues in Resonate which were causing OAuth to not function properly after addition of automatic build workflows. Related to #504
PR #524	Implement Friends and Direct calling functionality	This PR Implements all functionality related to Adding friends on the Resonate App and Direct Calls between friends. Fixes #523
PR #526	fix: Update Test Workflow triggers	This PR modifies the triggers for the automated tests workflow to allow for manual dispatch and dispatch on

Pull Request #	Pull Request Title	Pull Request Contributions
		updates to PRs. Part of #508
PR #529	Added Meilisearch Integration for search	This PR Adds support for Meilisearch which is the new search engine that Resonate will use to improve performance Fixes #528
PR #535	Add support for 16KB Page Size requirements in Android	This PR Adds support for 16 KB Memory Page Size requirements introduced by Android Fixes #533
PR #536	chore: Bump app version to 2.0.0+1 for v2 release	This PR Bumps the version of the app in Pubspect to 2.0.0 for v2 release in the play store Fixes #533
PR #537	Release v2 of Resonate App with all recent changes	This PR merges all recent changes over the past few months into master to deploy v2 of Resonate App to the Play Store Fixes #533
PR #538	chore: bump build number	This PR Bumps the build number of the app Fixes #533
PR #554	Implement better Host Moderation Controls and User Reporting	This PR Implements User Reporting functionality and improves existing Host Moderation controls. Fixes #553
PR #563	Add Live Chapters Feature	This PR Implements the Live Chapters feature. Start a Live Chapter Automatically Record and upload the audio file Locally transcribe to create lyrics via OpenAI Whisper Fixes #562
PR #565	various bugfixes	This PR fixes bugs related to #523 and #562
PR #564	Deploy v2.5 of Resonate App	This PR merges previous changes into main and deploys v2.5 of Resonate which include the new features of Improved Host Moderation Controls and Live Chapters with automated transcriptions.

PRs Merged after Mid Sem Eval (Backend Repo)

Pull Request #	Pull Request Title	Pull Request Contributions
PR #92	Add backend configuration for Creator Follow functionality	This PR adds backend support for the functionality detailed in AOSSIE-Org/Resonate#511 and is related to AOSSIE-Org/Resonate#512
PR #93	Implement backend config for Friends and Direct Calling functionalities	This PR Implements support for Friends and direct calling functionalities alongside AOSSIE-Org/Resonate#524
PR #96	Add Meilisearch integration	This PR adds backend support for AOSSIE-Org/Resonate#528
PR #99	Add Backend Configuration for Host Moderation features	This PR adds backend configurations related to AOSSIE-Org/Resonate#554 and AOSSIE-Org/Resonate#553
PR #107	feat: Add backend configuration for live chapters	This PR adds Backend Configuration pertaining to AOSSIE-Org/Resonate#563 and AOSSIE-Org/Resonate#562

Deliverables that could not be completed and lie out of the scope

Implementation of TTS and STT Accessibility features

The proposal included a plan to integrate TTS and STT features in the rooms for accessibility so that specially abled people could use Resonate. However, due to technical limitations, this feature could not be implemented at this time. A core requirement for this feature was the ability to get the Audio Stream from a LiveKit room and also be able to send a custom Audio Stream to the Room as input. The LiveKit Flutter SDK uses flutter_webrtc under the hood for all interactions with the native platforms, and as a result flutter_webrtc would have been the source of this stream. Now, although Sink APIs for AV Processing on the Native Side were added to flutter_webrtc in this PR: <https://github.com/flutter-webrtc/flutter-webrtc/pull/1700>, The ffis to access this Native Code from Dart were not yet implemented, and their progress is tracked under this issue: <https://github.com/flutter-webrtc/flutter-webrtc/issues/959>. As there is no way currently to get or insert

Audio Streams into the LiveKit rooms, this feature was unable to be implemented.

I did try to implement this feature using Loopback Audio, i.e: playing the room's audio normally from the speaker, while capturing it from the mic and then transcribing the captured audio, and while this approach would have "worked", it was unreliable and janky, and would have been very low quality. As Resonate was actively being updated even on it's Play Store deployment, I decided to not go ahead with the Loopback approach, as it would have been implementing the feature for the sake of it.

However, in my work on the Live Chapters feature, I have setup a controller for On Device transcription using OpenAI's Whisper Base, and there is example usage as well, which was my usage for implementing the Live Chapter Transcription. So future contributors have something to build upon once the changes in the upstream flutter_webrtc package are implemented. I myself will continue to actively search for alternate approaches even after GSoC and try my best to implement this feature whenever and however it becomes technically feasible.

For using and experiencing the features implemented by me as part of GSoC 2025, please download the Resonate App at https://play.google.com/store/apps/details?id=com.resonate.resonate&pcampaignid=web_share.

Conclusion

As Part of my GSoC 2025 Project, I added numerous key features to Resonate and also added some Quality of Life Changes for Maintainers and Contributors working on Resonate. I also acted as Repo Maintainer throughout my Project Tenure and I wholeheartedly plan to continue contributing, both to OSS and Resonate and AOSSIE even after GSoC. I thank Google and AOSSIE for giving me this chance to learn so much and contribute to OSS over the Summer, and the skills I gained here will be indispensable in my future life.

Thank you for the Best Summer Ever! 🎉🎊🎁